

Lecture 1: Introduction to Algorithms and their Limitations

CS 1200 | January 26, 2026 | Professor Anshu

How to read these notes. **Boxed** content is what was on the board: a **blue box** = captured in the slide transcription (ground truth); a **red dashed box** titled “**GENERATED**” = the room’s fixed camera missed that board, so the content is reconstructed from the transcript and should be verified. Everything outside the boxes is a structured summary of what Professor Anshu *said*.

Logistics

- Course website (Spring 26 section) is the hub; syllabus is long — read it.
 - **Grading scale:** N / L / R- / R / R+ (no numeric grades); revision of submitted work is possible.
 - **Participation is weighted like psets and exams** — in-class discussion and sender-receiver exercises are prioritized.
 - ps0 due Saturday; sections/office hours announced on Ed; textbook on Perusall.
 - Fifth or sixth offering; the course is “still evolving,” so feedback on the textbook is actively invited.
-

What is an Algorithm?

Professor Anshu opened by **collecting student answers** rather than defining the term:

On the board

- Steps to solve a problem
- A procedure: given inputs $\rightarrow$ outputs
- A simplification of a “real-world way to solve a problem”
- A methodology to manage a data set

His framing

- “There is no single correct answer” — every answer is “pointing in the right direction.”
 - Each one leans on a word that *isn’t yet precise*: **problem**, **procedure**, **methodology**.
 - **Making those precise is much of the course.** Pinning down “what is an algorithm” mathematically takes until Unit 2.
-

Course Structure

On the board

Unit	Topic
1	Storing & Searching

2	Computational Models
3	Graph & Logic Algorithms
4 / 5	Computational Complexity & Uncomputability

- Units 4 and 5 are “quite related.”
 - The whole course is **proofs-heavy**: “we’ll leave as little as possible under the rug” — though not Bertrand-Russell-level (we won’t re-derive what a number is).
 - Class size ~53.
-

Unit 1 — Goals

On the board

1. Abstracting a computational problem
2. Proving an algorithm solves a problem
3. Analyzing and comparing efficiency
4. Formalizing “what is an algorithm”

- “Faster is always better, given how impatient humans are.”
 - Goals 1–2 appear **today**; goal 2 recurs all semester; goals 3–4 fill Unit 1 and Unit 2.
 - He repeatedly pointed to the **Hesterberg–Vadhan textbook** for the rigorous proofs he only sketches in class.
-

Motivating Example: Keyword Search on Google

Professor Anshu picked Google search as “arguably the most impactful algorithm of the past couple of decades.” Classic search is three steps:

On the board

1. **PageRank** (precomputed): $\text{PR}(\text{url}) \in [0, 1]$ for every url on the web.
2. **Keyword search**: given a word w , form $S_w = \{\text{all urls containing } w\}$.
3. **Sorted output**: return the urls in S_w in **decreasing** order of $\text{PR}(\cdot)$.

- **Steps 1 and 2 are out of scope** — PageRank is its own algorithm; keyword search is a data-structures topic.
 - **Step 3 — sorting — is today’s subject.**
 - Higher PageRank ranks higher on the page (“if you’ve ever clicked to page 10 or 11 of Google results...”).
-

The Sorting Problem

The **first formal definition of a computational problem** in the course.

On the board

Input: an array $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$, where each key $K_i \in \mathbb{R}$ and each V_i is an arbitrary value.

Output: an array $A' = ((K'_0, V'_0), \dots, (K'_{n-1}, V'_{n-1}))$ that is a **valid sort** of A :

1. $K'_0 \leq K'_1 \leq \dots \leq K'_{n-1}$ (*keys ascending*)
2. $\exists$ a permutation $\pi : [n] \rightarrow [n]$ with $(K'_i, V'_i) = (K_{\pi(i)}, V_{\pi(i)})$ (*output is a rearrangement of the input*)

where $[n] = \{0, 1, \dots, n-1\}$.

- He flagged the notation as “what computer scientists like to use” and invited students to stop him if confused.
- **Values can be anything** — “cats and dogs.” The algorithm only looks at keys.
- **Computer scientists are scared of real numbers.** Real machines manipulate only rationals, so $K_i \in \mathbb{R}$ is an idealization; later (representing inputs) keys become rationals approximating reals. *Foreshadows the Word-RAM / representation discussion.*

Applying the Sorting Abstraction to Google Search

Run as a **2-minute pair exercise** (“turn to your neighbor”): express Google’s step 3 in the sorting abstraction.

On the board

From S_w , set for each url:

$$K_i = 1 - \text{PR}(\text{url}_i), \quad V_i = \text{url}_i.$$

Sorting in ascending order of K_i yields urls in **decreasing** PageRank order.

- The catch students hit: sorting is *ascending*, but we want *highest* PageRank first.
- Fix: $K_i = 1 - \text{PR}$ (or $-\text{PR}$ — “doesn’t matter, keys can be any real number”).
- Payoff of abstracting: one definition now covers Google, database queries, “anything you want to sort.”

Algorithm 1 — Exhaustive Search Sort (SS)

GENERATED — reconstructed; board not captured on camera

SS(A):

1. For every permutation $\pi : [n] \rightarrow [n]$, if $K_{\pi(0)} \leq K_{\pi(1)} \leq \dots \leq K_{\pi(n-1)}$, return the permuted array $((K_{\pi(0)}, V_{\pi(0)}), \dots)$.
2. Otherwise (no permutation sorts), return $\perp$ — a special “no answer” symbol.

Theorem (correctness of SS). If a valid sort of A exists, then **SS** outputs a valid sort of A .

Proof. Suppose A' is a valid sort. By definition there is a permutation π^* with $K_{\pi^*(0)} \leq \dots \leq K_{\pi^*(n-1)}$. **SS** enumerates *all* permutations, so it reaches π^* (or an earlier sorting permutation) and returns there by the if-condition. ■

Why start here

- Deliberately the “silliest” algorithm — to model a habit: **write the obvious brute force first, then refine.**
- The theorem says “**if** a valid sort exists.” Proving one *always* exists is “not as easy as it seems,” so he deferred it (insertion sort discharges it for free).
- **What’s wrong with SS?** A student answered: inefficient — you may scan all $n!$ permutations, and the one you need could be last. “Too long” for something as elementary as sorting. Runtime analysis comes next lecture; this motivates insertion sort.

Algorithm 2 — Insertion Sort (IS)

On the board

IS(A): for $i = 0$ to $n - 1$:

1. Find the **smallest** $j \leq i$ such that $K_j \geq K_i$ (the key at position j is $\geq$ the key at position i).
2. Insert $A[i]$ at position j , shifting $A[j \dots i - 1]$ up to $A[j + 1 \dots i]$.

- **Intuition:** “start from the beginning, consume the next element, slot it into the sorted part, move on.”
- He sold IS on **two** grounds, not just speed:
 - Much more efficient than SS.
 - Its correctness proof **also proves a valid sort always exists** — no “if it exists” caveat. “Something very nice is happening”: the efficient algorithm also explains *why* sorting works.
- **Student question (common confusion):** why $\geq$ and not $>$? If $A[i]$ is already the largest so far, no smaller j qualifies, so $j = i$ and the element stays put. The $\geq$ lets one clause also cover “don’t move it.”

Worked example

On the board

Input: $(6, a), (2, b), (1, c), (9, d)$

i	Array state
0	$(6, a), (2, b), (1, c), (9, d)$
1	$(2, b), (6, a), (1, c), (9, d)$
2	$(1, c), (2, b), (6, a), (9, d)$
3	$(1, c), (2, b), (6, a), (9, d)$

- Walked through each i by hand, asking after each step whether the prefix was sorted.
- With this input the last key 9 is already largest, so step 3 leaves the array unchanged.

Insertion Sort — Correctness via a Loop Invariant

GENERATED — reconstructed; board not captured on camera

Theorem. Insertion sort solves the sorting problem (no “if it exists” needed).

Setup. Let $A^{(i)}$ denote the array **right before** step i (so $A^{(0)}$ is the input).

Loop invariant $P(i)$: before step i , the prefix $A^{(i)}[0 \dots i - 1]$ is sorted, and the suffix $A^{(i)}[i \dots n - 1]$ is unchanged from the input.

Base case $P(0)$: trivial — the sorted prefix is empty and the array is still the input.

Inductive step: assume $P(i)$; show $P(i + 1)$. Step i finds the smallest j with $K_j \geq K_i$, inserts $A[i]$ before position j , and shifts the block up. The prefix stays sorted: everything left of j is $< K_i$ (since j is the *smallest* index whose key is $\geq K_i$), and everything from j onward was $\geq K_i$ and keeps its order. The suffix past i is untouched. Hence $A^{(i+1)}[0 \dots i]$ is sorted. By induction $P(n)$ holds, so the final array is fully sorted. ■

- This is the course’s **first real induction / loop-invariant argument**; he noted students are likely proving induction on the current pset.
- He **openly fumbled the index ranges** at the board (“nervous about missing these i and $i + 1$ ”) and pushed on — an honest model of how proofs get debugged.
- Gave a **picture proof**; sent students to the textbook for the formal version. Ran slightly over time.

Where to read

Textbook (Hesterberg–Vadhan, on Perusall) §§1.1–1.5 — formal versions of both proofs sketched today. **Next lecture (Wed):** runtime analysis.